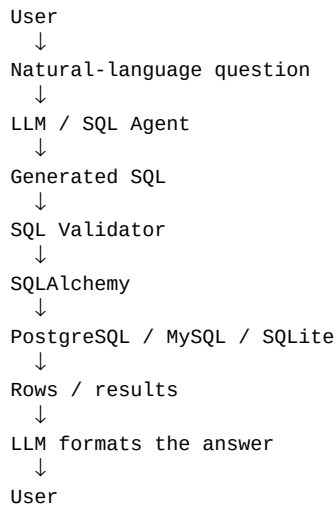


SQLAlchemy Crash Course

Focused on building a SQL Agent that generates and executes SQL

This guide deliberately focuses on the SQLAlchemy features you actually need for a natural-language SQL agent. You do not need to master the entire SQLAlchemy ecosystem before starting the project.

1. Target Architecture



2. Install SQLAlchemy

```
pip install sqlalchemy

# PostgreSQL
pip install psycopg
```

For PostgreSQL, a typical connection URL is:

```
DATABASE_URL = "postgresql+psycopg://user:password@localhost:5432/mydb"
```

For a lightweight local project:

```
DATABASE_URL = "sqlite:///database.db"
```

3. The Engine

The Engine is SQLAlchemy's main entry point to the database. Normally you create one engine and reuse it.

```
from sqlalchemy import create_engine

engine = create_engine(
    "sqlite:///database.db"
)
```

4. Test the Connection

```
from sqlalchemy import create_engine, text

engine = create_engine("sqlite:///database.db")

with engine.connect() as conn:
    result = conn.execute(text("SELECT 1"))
    print(result.scalar())
```

Important: raw SQL is normally wrapped with **text()**.

5. Execute SQL

```
from sqlalchemy import text

with engine.connect() as conn:
    result = conn.execute(
        text("SELECT * FROM users")
    )
```

The returned Result object contains the query output.

6. Reading Results

fetchall()

```
with engine.connect() as conn:
    result = conn.execute(text("SELECT * FROM users"))
    rows = result.fetchall()

print(rows)
```

first()

```
row = result.first()
```

scalar()

```
with engine.connect() as conn:
    result = conn.execute(
        text("SELECT COUNT(*) FROM users")
    )
    count = result.scalar()

print(count)
```

Accessing columns

```
for row in result:
    print(row.id)
    print(row.name)
```

For an agent, converting rows to dictionaries is especially convenient:

```
for row in result:
    print(dict(row._mapping))
```

7. The Core Executor You Need

```
from sqlalchemy import text

def execute_sql(sql: str):
    with engine.connect() as conn:
        result = conn.execute(text(sql))

        rows = [
            dict(row._mapping)
            for row in result
        ]

    return rows
```

This small function is essentially your first SQLAlchemy execution layer.

8. Parameters

Use bound parameters instead of constructing SQL with string interpolation.

```
# Good
sql = text("""
```

```

SELECT *
FROM users
WHERE id = :user_id
""")

with engine.connect() as conn:
    result = conn.execute(
        sql,
        {"user_id": 5}
    )

```

Avoid building SQL values by directly concatenating untrusted input.

9. Transactions

For read-only queries, use `connect()`. For modifications, use a transaction with `begin()`.

```

# Reading
with engine.connect() as conn:
    result = conn.execute(
        text("SELECT * FROM users")
    )

# Modification
with engine.begin() as conn:
    conn.execute(
        text("""
            UPDATE users
            SET name = :name
            WHERE id = :id
        """),
        {"name": "Yazid", "id": 1}
    )

```

10. Database Inspection

Your LLM needs to know what tables and columns exist before it can reliably generate SQL. SQLAlchemy's `inspect()` is very useful for this.

```

from sqlalchemy import inspect

inspector = inspect(engine)

tables = inspector.get_table_names()
print(tables)

```

Get columns

```

columns = inspector.get_columns("employees")

for column in columns:
    print(column["name"], column["type"])

```

Get foreign keys

```

foreign_keys = inspector.get_foreign_keys("employees")
print(foreign_keys)

```

11. Build Schema Context for the LLM

```

from sqlalchemy import inspect

def get_schema(engine):
    inspector = inspect(engine)
    schema = {}

    for table in inspector.get_table_names():
        columns = inspector.get_columns(table)

        schema[table] = [

```

```

        {
            "name": column["name"],
            "type": str(column["type"])
        }
        for column in columns
    ]

    return schema

```

You can then give the LLM structured information such as:

```

employees:
- id INTEGER
- name VARCHAR
- salary FLOAT
- department VARCHAR

departments:
- id INTEGER
- name VARCHAR

```

12. Read-Only SQL Agent

For your first version, make the agent read-only. Do not let an LLM freely execute DROP, DELETE, or UPDATE statements.

```

def execute_sql(sql: str):
    sql_clean = sql.strip().lower()

    if not sql_clean.startswith("select"):
        raise ValueError(
            "Only SELECT queries are allowed"
        )

    with engine.connect() as conn:
        result = conn.execute(text(sql))

    return [
        dict(row._mapping)
        for row in result
    ]

```

Important: this is only a basic application-level guard, not a complete security boundary. For production, use a database account with read-only permissions and add proper SQL validation.

13. Protect Against Huge Results

An LLM might generate SELECT * against a very large table. For your first agent, instruct it to use LIMIT for raw row queries.

```

SELECT *
FROM employees
LIMIT 20;

```

You can also enforce result-size limits in your execution layer.

14. SQLAlchemy Core vs ORM

```

SQLAlchemy
■■■ Core ← focus here first
■■■ ORM  ← learn later

```

For a natural-language SQL agent, SQLAlchemy Core is directly relevant because your agent already produces SQL strings. ORM is more useful when you are building conventional application models and CRUD logic.

15. FastAPI Integration

A simple first architecture could look like this:

```
sql-agent/  
├── app/  
│   ├── main.py  
│   ├── database.py  
│   ├── executor.py  
│   ├── schema.py  
│   └── agent.py  
└── requirements.txt
```

database.py

```
from sqlalchemy import create_engine  
  
DATABASE_URL = "sqlite:///database.db"  
  
engine = create_engine(DATABASE_URL)
```

executor.py

```
from sqlalchemy import text  
from .database import engine  
  
def execute_sql(sql: str):  
    with engine.connect() as conn:  
        result = conn.execute(text(sql))  
  
        return [  
            dict(row._mapping)  
            for row in result  
        ]
```

main.py

```
from fastapi import FastAPI  
from .executor import execute_sql  
  
app = FastAPI()  
  
@app.post("/sql")  
def run_sql(sql: str):  
    result = execute_sql(sql)  
  
    return {"rows": result}
```

16. Build the Agent in Stages

Version 1
User SQL → SQLAlchemy → Database → Results

Version 2
Database → inspect() → Schema → LLM context

Version 3
Question → LLM → SQL → SQLAlchemy → Results

Version 4
LLM SQL → Validator → SQLAlchemy

Version 5
SQL error → LLM sees error → Fix SQL → Execute again

17. Agentic Error Correction

A useful next step is to let the agent recover from SQL errors.

User:
"What are the top 5 products?"

LLM:

```
SELECT product, revenuee
FROM sales
ORDER BY revenuee DESC
LIMIT 5
```

Database:
column "revenuee" does not exist

Agent:
inspect schema / analyze error

LLM:
SELECT product, revenue
FROM sales
ORDER BY revenue DESC
LIMIT 5

Database:
SUCCESS

This turns the project from a simple text-to-SQL demo into a more interesting agentic workflow.

18. SQLAlchemy Cheat Sheet

```
from sqlalchemy import create_engine, text, inspect

# Engine
engine = create_engine(DATABASE_URL)

# Connection
with engine.connect() as conn:

# Execute
result = conn.execute(text(sql))

# Parameters
conn.execute(
    text("SELECT * FROM users WHERE id = :id"),
    {"id": 5}
)

# Rows
result.fetchall()

# First row
result.first()

# One value
result.scalar()

# Dictionary
dict(row._mapping)

# Inspect database
inspector = inspect(engine)

# Tables
inspector.get_table_names()

# Columns
inspector.get_columns("users")

# Foreign keys
inspector.get_foreign_keys("users")

# Transaction
with engine.begin() as conn:
    ...
```

19. What to Learn vs What to Skip

HIGH PRIORITY

- ✓ create_engine()
- ✓ connect()
- ✓ text()
- ✓ execute()
- ✓ Result / Row
- ✓ fetchall()
- ✓ first()
- ✓ scalar()
- ✓ row._mapping
- ✓ bound parameters
- ✓ transactions
- ✓ inspect()
- ✓ get_table_names()
- ✓ get_columns()
- ✓ get_foreign_keys()

LOW PRIORITY FOR YOUR FIRST AGENT

- Advanced ORM
- Complex relationship mappings
- Declarative model design
- Advanced ORM querying
- SQLAlchemy internals

20. Recommended 5-Day Project Path

DAY 1

Engine + connection + text() + execute() + results

DAY 2

Parameters + transactions + inspect() + schema extraction

DAY 3

Build a clean SQL executor

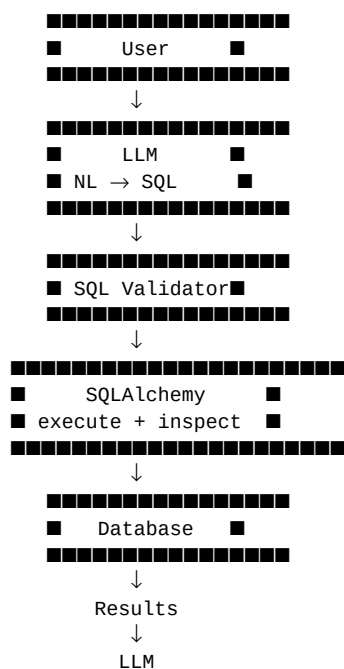
DAY 4

Connect LLM → SQLAlchemy

DAY 5+

SQL validation
Schema-aware prompting
LIMIT / result protection
Error correction
Agent loop

Final Mental Model



↓
Natural-language
answer

Core takeaway: SQLAlchemy is your database execution and inspection layer. The agent itself is the loop around it: understand the question → generate SQL → validate → execute → inspect errors/results → explain the result.